

دانشگاه صنعتی خواجه نصیرالدین طوسی

عنوان درس: تشخیص الگو در تصاویر سنجش از دور

تمرین چهارم: طبقه‌بندی تصویر با روش های RF، XGBoost و LightGBM

نام و نام خانوادگی: شایان کشیری

فهرست

مقدمه.....	۳
۱- google earth engine.....	۳
۱-۱- تهیه داده‌ها و تصاویر.....	۳
۱-۲- ورود داده‌ها و تصویر.....	۵
۲- کلاسه‌بندی Random Forest.....	۷
۳- کلاسه‌بندی XGBoost.....	۱۳
۴- کلاسه‌بندی LightGBM.....	۱۹
۵- مقایسه SVM و Random forest.....	۲۵
۶- بررسی overfitting.....	۳۲

مقدمه

طبقه‌بندی یکی از مهم‌ترین شاخه‌های یادگیری ماشین است که هدف آن پیش‌بینی برچسب یا کلاس یک نمونه بر اساس ویژگی‌هایش می‌باشد. در این روش، مدل با استفاده از داده‌های آموزشی که کلاس آن‌ها مشخص است، یاد می‌گیرد تا نمونه‌های جدید را به کلاس‌های مناسب تخصیص دهد. الگوریتم‌های رایج طبقه‌بندی شامل SVM، Random Forest، XGBoost و LightGBM هستند. این روش در زمینه‌های مختلفی مانند تشخیص تصویر، پزشکی، سنجش از دور و تحلیل داده‌های مالی کاربرد دارد. دقت و قابلیت تعمیم مدل از مهم‌ترین معیارهای ارزیابی آن است. برای سنجش عملکرد معمولاً از confusion matrix، دقت کلی (Overall Accuracy)، user accuracy، producer accuracy و kappa استفاده می‌شود. انتخاب الگوریتم مناسب و بهینه‌سازی پارامترها نقش مهمی در موفقیت طبقه‌بندی دارد.

۱- google earth engine

۱-۱- تهیه داده‌ها و تصاویر

قبل از شروع نوشتن کدهای سه الگوریتم طبقه‌بندی، در google earth engine منطقه‌ای را برای کلاسه‌بندی انتخاب می‌کنیم. سپس برای این منطقه سه دسته در سه کلاس آب، شهری و پوشش گیاهی داده جمع‌آوری می‌کنیم.

۱. Low: تعداد داده‌هایی که جمع‌آوری می‌کنیم، بین ۵۰ تا ۸۰ داده است (برای هر سه کلاس ۵۰ تا).

۲. Medium: تعداد داده‌هایی که جمع‌آوری می‌کنیم، ۱۰۰ داده است (برای هر کلاس ۱۰۰ تا).

۳. High: تعداد داده‌هایی که جمع‌آوری می‌کنیم، بین ۲۰۰ تا ۳۰۰ داده است (برای کلاس‌های آب و پوشش

گیاهی ۲۰۰ و برای شهری ۲۸۰ تا).

حال کد خود را برای دانلود این داده‌ها طبق کد زیر می‌نویسیم (شکل ۱-۱).

```
i 1 Map.centerObject(geometry)
2 var s2 = ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
3   .filterDate('2022-06-01', '2022-09-01')
4   .filterBounds(geometry)
5   .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 1));
6   print('number of images:', s2.size());
7
8 var Image = s2.first();
9
10 print('date of image:', Image.date());
11 print('cloud cover:', Image.get('CLOUDY_PIXEL_PERCENTAGE'))
12 print('id:', Image.get('system:id'));
13
14 var image=Image.select(['B2', 'B3', 'B4', 'B5', 'B6', 'B7', 'B8']).clip(geometry)
15
16 Map.centerObject(geometry, 14)
17 Map.addLayer(geometry, {color: 'blue'}, 'ROI')
18 Map.addLayer(image, {bands: ['B4', 'B3', 'B2'], min: 0, max: 3000}, 'sentinel 2rgb')
19 var training_data = water.merge(urban).merge(vegetation)
20
21 var bands=['B2', 'B3', 'B4', 'B5', 'B6', 'B7', 'B8']
22 var training= image.select(bands).sampleRegions({
23   collection: training_data,
24   properties: ['landcover'],
25   scale: 10 })
26 print('train data:', training)
27
28 Export.image.toDrive({
29   image: image,
30   description: 'sentinel_2A',
31   folder: 'sisangan',
32   region: geometry,
33   scale: 10,
34   fileFormat: 'GeoTIFF',
35   maxPixels: 1e9
36 });
37
38 Export.table.toDrive({
39   collection: training,
40   description: 'sentinel_2A_training',
41   folder: 'GEE_Exports',
42   fileNamePrefix: 'training_data_sisangan',
43   fileFormat: 'CSV'
44 });
```

شکل ۱-۱) کد دانلود تصویر و داده‌ها

پس از جمع‌آوری این سه دسته داده، تصویر خود که دارای باندهای ۲ (blue)، ۳ (green)، ۴ (red)، ۵ (nir)، ۶ (swir-1)، ۷ (swir2) است (شکل ۱-۲) را وارد فضای پایتون کرده تا کلاسه‌بندی‌های خود را با استفاده از سطوح داده مختلف انجام دهیم.

۱-۲- ورود داده‌ها و تصویر

در مرحله اول، تمام باندهای تصویر خود را خوانده و فرمت آن را به ترتیب سطر، ستون و باند تنظیم می‌کنیم.

```
with rasterio.open(r'C:\Users\Administrator\Downloads\sentinel_2A (2).tif') as src:
    img=src.read()
    profile=src.profile
    img=img.transpose(1,2,0).astype(float)
    rows, cols, bands = img.shape
```

سپس داده‌هایی که جمع‌آوری کردیم را وارد می‌کنیم.

در این مرحله برای X، مقادیر تمام باندها را برای هر داده خوانده و برای مقدار Y، نوع پوشش را در نظر می‌گیریم.

```
data=pd.read_csv(r'C:\Users\Administrator\Downloads\training_data_sisangan(3).csv')
x=data[['B2','B3','B4','B5','B6','B7','B8']].values.astype(float)
y=data['landcover'].values.astype(int)
```

در مرحله بعد، باید مقادیر باندها را که برای داده‌های خود خواندیم، نرمال کنیم.

```
if x.max()>100:  
    x=x/10000  
    img_norm=img/10000  
else:  
    img_norm=img.copy()
```

در گام بعد، داده‌های خود را به دو دسته آموزشی و تست تقسیم می‌کنیم.

```
x_train, x_test, y_train, y_test=train_test_split(  
    x , y , test_size=0.3 , stratify=y , random_state=42)
```

Startify=y به این معناست که تقسیم داده‌ها به طور تصادفی با حفظ توزیع کلاس‌ها انجام می‌شود. یعنی کلاسی که تعداد داده‌های آموزشی بیشتری نسبت به دیگری دارد، در داده‌های تست و آموزشی نیز به همان نسبت تعدادش بیشتر است. از random state نیز به این دلیل استفاده شده‌است که تقسیم داده‌ها در هر مرحله ثابت باشد.

پس از آن، داده‌های خود را استاندارد می‌کنیم تا دارای میانگین ۰ و انحراف معیار ۱ باشند.

```
scaler=StandardScaler().fit(x_train)  
x_train_s=scaler.transform(x_train)  
x_test_s=scaler.transform(x_test)
```

تابع `fit()` این مقادیر را برای x های آموزشی ما ذخیره می‌کند و این استانداردسازی روی این داده‌ها اعمال می‌شود. داده‌های تست نیز استاندارد می‌شوند اما از تابع `fit()` در آن استفاده نمی‌کنیم. چون این داده‌ها نباید در فرایند آموزش ما وارد شوند.

در گام بعد، تمام باندهای تصویر خود را یکپارچه می‌کنیم.

```
flat=img_norm.reshape(-1, bands)
flat_s = scaler.transform(flat)
```

حال داده‌ها و تصویر ما آماده انجام کلاسه‌بندی هستند.

۲- کلاسه‌بندی Random Forest

Random Forest، یک الگوریتم یادگیری ماشین مبتنی بر decision tree است که از ترکیب چندین درخت تصمیم برای بهبود دقت و کاهش overfitting استفاده می‌کند. هر درخت با داده‌های تصادفی و زیرمجموعه‌ای تصادفی از ویژگی‌ها آموزش داده می‌شود. در نهایت، انتخاب نهایی از طریق رای اکثریت یا میانگین تعیین می‌شود. مهم‌ترین مزیت آن کاهش overfitting و قابلیت کار با داده‌های نویزدار و پیچیده است.

برای انجام این کلاسه‌بندی از تابع مخصوص استفاده کرده و این کلاسه‌بندی را روی داده‌های آموزشی خود

اعمال می‌کنیم. برای کلاسه‌بندی از تابع `RandomForestClassifier()` استفاده می‌کنیم.

```
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(x_train_s, y_train)
```

در گام بعد، این آموزش را روی داده‌های تست خود آزمایش می‌کنیم

```
y_pred_rf = rf.predict(x_test_s)
```

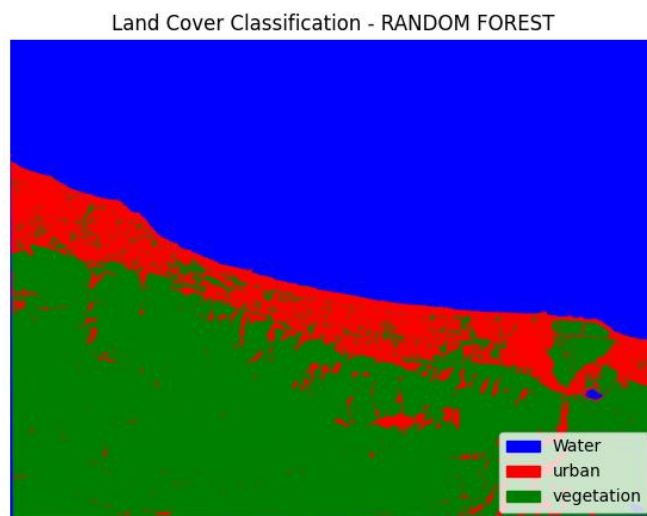
در مرحله آخر اعمال الگوریتم، کلاسه‌بندی تصویر خود را انجام می‌دهیم (شکل‌های ۲-۱ و ۲-۲ و ۲-۳).

```
pred_rf_full = rf.predict(flat_s).reshape(rows, cols)
```

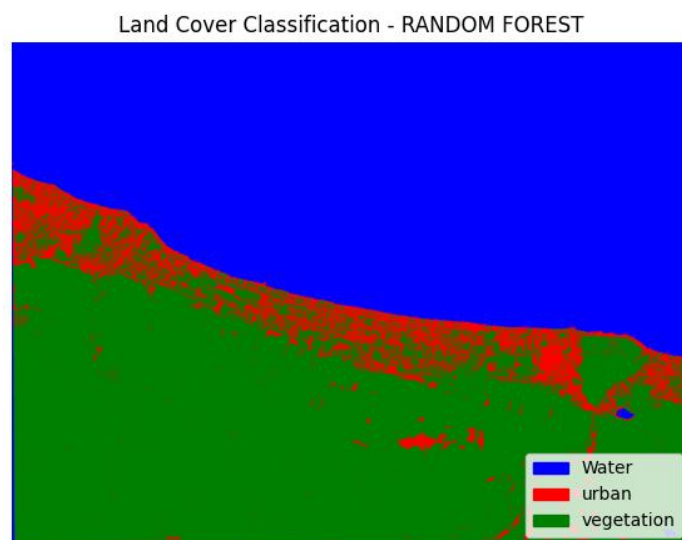
پارامترهایی که در Random Forest تعریف می‌کنیم:

`n_estimators`: تعداد درخت‌هایی که در مدل ساخته می‌شوند.

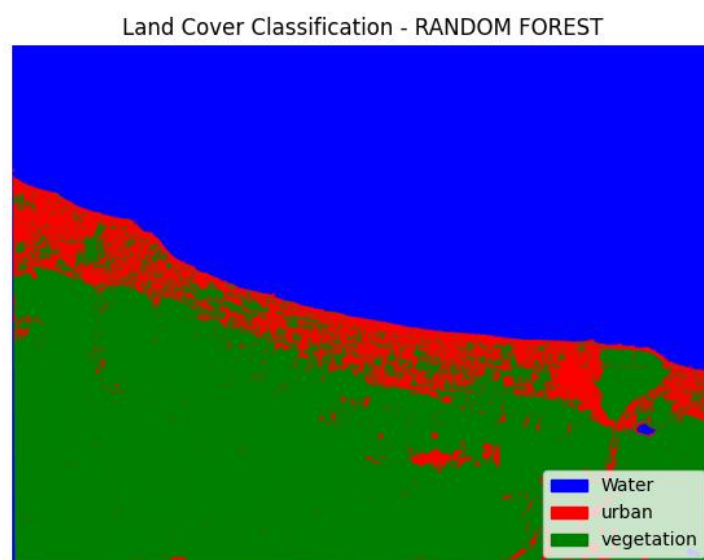
`random_state`: تعداد تکرارهای مدل.



شکل ۲-۱) کلاسه‌بندی Random Forest با داده‌های سطح Low



شکل ۲-۲) کلاسه‌بندی Random Forest با داده‌های سطح Medium



شکل ۲-۳) کلاسه‌بندی Random Forest با داده‌های سطح High

مقدار کاپا، گزارش کلاسه بندی و ماتریس کانفیوژن ما برای داده‌های سطح Low، Medium و High به ترتیب براساس شکل‌های ۲-۴، ۲-۵ و ۲-۶ و جدول ۲-۱، ۲-۲ و ۲-۳ هستند.

Kappa random forest: 0.8666666666666667				
classification report random forest:				
	precision	recall	f1-score	support
1	1.00	1.00	1.00	15
2	0.87	0.87	0.87	15
3	0.87	0.87	0.87	15
accuracy			0.91	45
macro avg	0.91	0.91	0.91	45
weighted avg	0.91	0.91	0.91	45

شکل ۲-۴) کاپا و گزارش کلاسه‌بندی Random Forest برای داده سطح Low

جدول ۲-۱) ماتریس کانفیوژن برای کلاسه‌بندی Random Forest برای داده‌های Low

۱۵	۰	۰
۰	۱۳	۲
۰	۲	۱۳

Kappa random forest: 0.9				
classification report random forest:				
	precision	recall	f1-score	support
1	1.00	1.00	1.00	30
2	0.93	0.87	0.90	30
3	0.88	0.93	0.90	30
accuracy			0.93	90
macro avg	0.93	0.93	0.93	90
weighted avg	0.93	0.93	0.93	90

شکل ۵-۲) کاپا و گزارش کلاسه‌بندی Random Forest برای داده سطح Medium

جدول ۲-۲) ماتریس کانفیوژن برای کلاسه‌بندی Random Forest برای داده‌های Medium

۳۰	۰	۰
۰	۲۶	۴
۰	۲	۲۸

Kappa random forest: 0.9177660510114336				
classification report random forest:				
	precision	recall	f1-score	support
1	0.98	1.00	0.99	60
2	0.92	0.95	0.94	84
3	0.95	0.88	0.91	60
accuracy			0.95	204
macro avg	0.95	0.95	0.95	204
weighted avg	0.95	0.95	0.95	204

شکل ۶-۲) کاپا و گزارش کلاسه‌بندی Random Forest برای داده سطح High

جدول ۲-۳) ماتریس کانفیوژن برای کلاسه‌بندی Random Forest برای داده‌های High

۶۰	۰	۰
۱	۸۰	۳
۰	۷	۵۳

بر اساس مقادیر ذکرشده در گزارش کلاسه‌بندی (precision همان user accuracy و recall همان producer accuracy است)، مقادیر زیر نیز برآورد شد (جدول ۲-۴).

جدول ۲-۴) پارامترهای طبقه‌بندی Random Forest

	OA	PA(water)	UA(water)	PA(urban)	UA(urban)	PA(veg)	UA(veg)
High	۰/۹۵	۱	۰/۹۸	۰/۹۵	۰/۹۲	۰/۸۸	۰/۹۵
Medium	۰/۹۳	۱	۱	۰/۸۷	۰/۹۳	۰/۹۳	۰/۸۸
Low	۰/۹۱	۱	۱	۰/۸۷	۰/۸۷	۰/۸۷	۰/۸۷

۳- کلاسه‌بندی XGBoost

XGBoost، یک الگوریتم یادگیری ماشین مبتنی بر Gradient Boosting است. این روش به صورت تک به تک درخت‌ها را می‌سازد و هر درخت تلاش می‌کند خطای درخت‌های قبلی را کاهش دهد. XGBoost با بهینه‌سازی پیشرفته و Regularization سرعت و دقت بالایی دارد و معمولاً از overfitting جلوگیری می‌کند. این الگوریتم توانایی کار با داده‌های بزرگ و پیچیده را دارد و مهم‌ترین مزیت آن، دقت بالا و قابلیت تنظیم پارامترهای متعدد برای بهبود عملکرد است.

برای اعمال الگوریتم XGBoost روال کار مانند الگوریتم Random Forest می‌باشد. اما تابعی که باید فرا بخوانیم، تابع مخصوص الگوریتم XGBoost است که XGBClassifier نام دارد.

نکته دیگری که باید رعایت کنیم این است که برای این الگوریتم، y های ما یعنی کلاس‌های ما باید از ۰ شروع شوند در حالی که داده‌های ما از ۱ شروع می‌شوند. برای این کار، ابتدا باید از مقادیر y های خود که ۱، ۲ و ۳ هستند یکی کم کنیم تا به ۰، ۱ و ۲ تبدیل شوند و الگوریتم قابل پیاده‌سازی باشد.

```
y_train_xgb = y_train - 1
y_test_xgb = y_test - 1
```

حال کلاسه‌بندی خود را انجام می‌دهیم (شکل‌های ۳-۱ و ۳-۲ و ۳-۳).

همانطور که در بالا گفته شد، مراحل آن درست مانند روش Random Forest بوده و فقط تابع ما متفاوت است.

```
xgb = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
```

```

subsample=0.8,
colsample_bytree=0.8,
objective='multi:softmax',
num_class=len(np.unique(y_train_xgb)),
eval_metric="mlogloss",
random_state=42
)

xgb.fit(x_train_s, y_train_xgb)
y_pred_xgb = xgb.predict(x_test_s)
pred_xgb_full = xgb.predict(flat_s).reshape(rows, cols)

```

پارامترهایی که باید برای این الگوریتم تعریف کنیم به شرح زیر است:

`n_estimators`: تعداد درخت‌هایی که در مدل ساخته می‌شوند.

`max_depth`: حداکثر عمق (شاخه) هر درخت.

`learning_rate`: نرخ یادگیری.

`subsample`: نسبت نمونه‌های آموزش که برای هر درخت استفاده می‌شود.

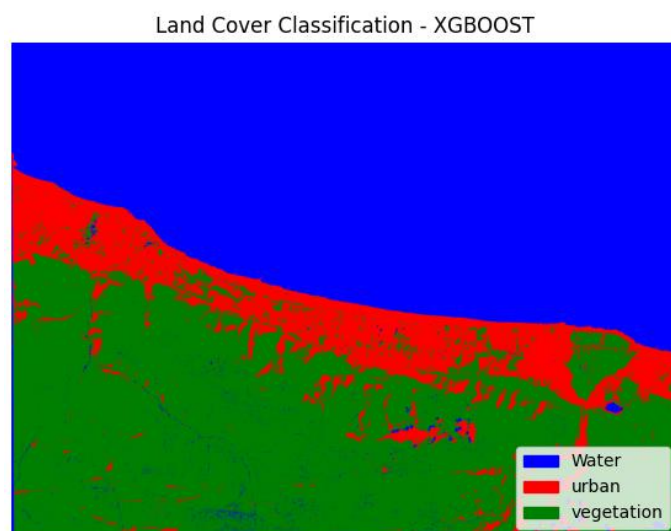
`colsample_bytree`: نسبت ویژگی‌ها که برای هر درخت استفاده می‌شود.

`objective`: تعیین می‌کند مدل یک مسئله چندکلاسه را پیش‌بینی می‌کند و خروجی کلاس‌ها را مستقیم می‌دهد.

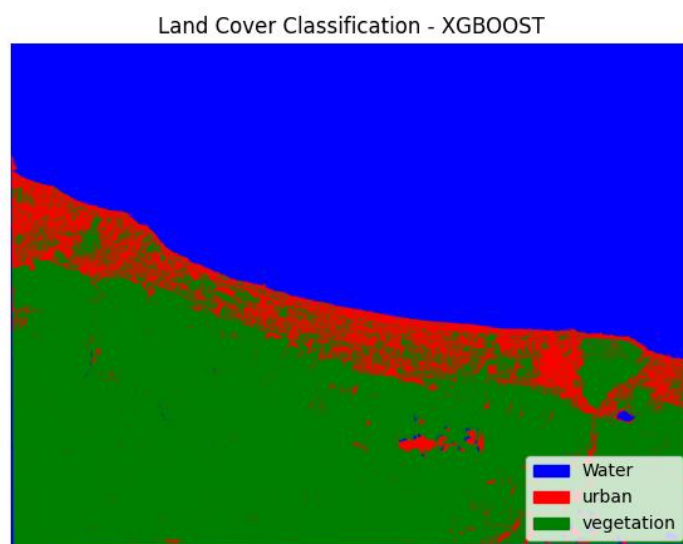
`num_class`: تعداد کلاس‌ها در مسئله چندکلاسه.

`eval_metric`: معیار ارزیابی مدل در طول آموزش.

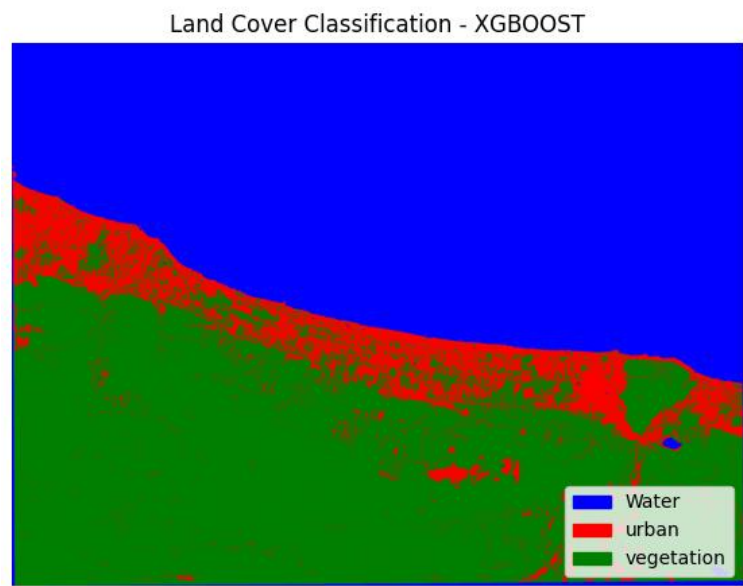
`random_state`: تعداد تکرارهای مدل.



شکل (۳-۱) کلاسه‌بندی XGBoost با داده‌های سطح Low



شکل (۳-۲) کلاسه‌بندی XGBoost با داده‌های سطح Medium



شکل ۳-۳) کلاسه‌بندی XGBoost با داده‌های سطح High

مقدار کاپا، گزارش کلاسه‌بندی و ماتریس کانفیوژن ما برای داده‌های سطح Low، Medium و High به‌ترتیب براساس شکل ۳-۴، ۳-۵ و ۳-۶ و جدول‌های ۳-۱، ۳-۲ و ۳-۳ هستند.

Kappa xgboost: 0.8666666666666667					
classification report xgbosst:					
	precision	recall	f1-score	support	
0	1.00	1.00	1.00	15	
1	0.87	0.87	0.87	15	
2	0.87	0.87	0.87	15	
accuracy			0.91	45	
macro avg	0.91	0.91	0.91	45	
weighted avg	0.91	0.91	0.91	45	

شکل ۳-۴) کاپا و گزارش کلاسه‌بندی XGBoost برای داده سطح Low

جدول ۳-۱) ماتریس کانفیوژن برای کلاسه‌بندی XGBoost برای داده‌های Low

۱۵	۰	۰
۰	۱۳	۲
۰	۲	۱۳

Kappa xgboost: 0.9				
classification report xgboost:				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	30
1	0.90	0.90	0.90	30
2	0.90	0.90	0.90	30
accuracy			0.93	90
macro avg	0.93	0.93	0.93	90
weighted avg	0.93	0.93	0.93	90

شکل ۳-۵) کاپا و گزارش کلاسه‌بندی XGBoost برای داده سطح Medium

جدول ۳-۲) ماتریس کانفیوژن برای کلاسه‌بندی XGBoost برای داده‌های Medium

۳۰	۰	۰
۰	۲۷	۳
۰	۳	۲۷

Kappa xgboost: 0.8954305799648506				
classification report xgbosst:				
	precision	recall	f1-score	support
0	0.98	1.00	0.99	60
1	0.91	0.93	0.92	84
2	0.91	0.87	0.89	60
accuracy			0.93	204
macro avg	0.93	0.93	0.93	204
weighted avg	0.93	0.93	0.93	204

شکل ۳-۶) کاپا و گزارش کلاسه‌بندی XGBoost برای داده سطح High

جدول ۳-۳) ماتریس کانفیوژن برای کلاسه‌بندی XGBoost برای داده‌های High

۶۰	۰	۰
۱	۷۸	۵
۰	۸	۵۲

بر اساس گزارش کلاسه‌بندی، مقادیر زیر برآورد شد (جدول ۳-۴).

جدول ۳-۴ پارامترهای طبقه‌بندی XGBoost

	OA	PA(water)	UA(water)	PA(urban)	UA(urban)	PA(veg)	UA(veg)
High	۰/۹۳	۱	۰/۹۸	۰/۹۳	۰/۹۱	۰/۸۷	۰/۹۱
Medium	۰/۹۳	۱	۱	۰/۹	۰/۹	۰/۹	۰/۹
Low	۰/۹۱	۱	۱	۰/۸۷	۰/۸۷	۰/۸۷	۰/۸۷

۴- کلاسه‌بندی LightGBM

LightGBM یک الگوریتم یادگیری ماشین مبتنی بر Gradient Boosting است که برای سرعت و مقیاس‌پذیری بهینه شده است. برخلاف روش‌های سنتی، LightGBM از تجزیه برگ‌محور (leaf-wise) به جای سطح‌محور (level-wise) برای ساخت درخت‌ها استفاده می‌کند که دقت بالاتری دارد. این الگوریتم می‌تواند با داده‌های بزرگ و ویژگی‌های زیاد به خوبی کار کند و مصرف حافظه کمی دارد. LightGBM از Regularization برای جلوگیری از overfitting استفاده می‌کند و پارامترهای زیادی برای بهینه‌سازی عملکرد دارد و مهم‌ترین مزیت آن سرعت بالا و دقت زیاد است.

در پیاده‌سازی الگوریتم LightGBM نیز روش ما مانند دو روش گذشته بوده و فقط تابع مورد استفاده ما فرق می‌کند که LGBMClassifier می‌باشد.

در این روش نیز باید مانند روش XGBoost، از مقادیر y خود که کلاس‌های ما هستند یکی کم کنیم تا به شکل ۰، ۱ و ۲ تبدیل شده و الگوریتم قابل اجرا باشد (شکل‌های ۴-۱ و ۴-۲ و ۴-۳).

```

y_train_lgbm = y_train - 1
y_test_lgbm = y_test - 1

lgbm = LGBMClassifier(
    n_estimators=200,
    learning_rate=0.1,
    random_state=42
)

lgbm.fit(x_train_s, y_train_lgbm)

y_pred_lgbm = lgbm.predict(x_test_s)

pred_lgbm_full = lgbm.predict(flat_s).reshape(rows, cols)

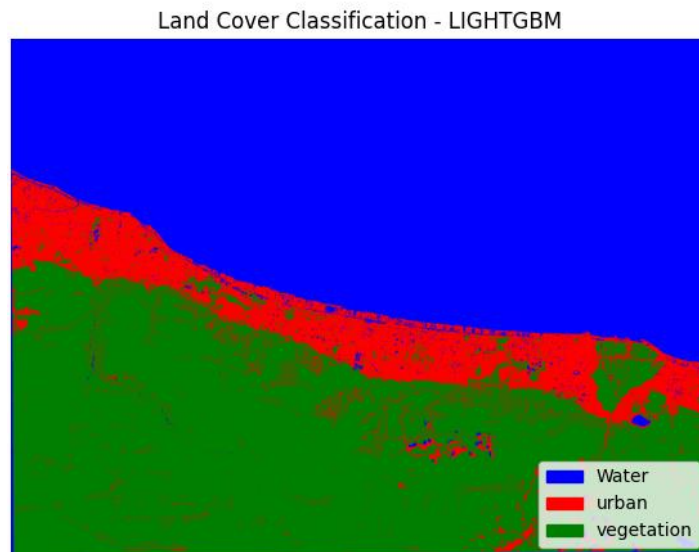
```

پارامترهای این الگوریتم طبق توضیحات زیر است:

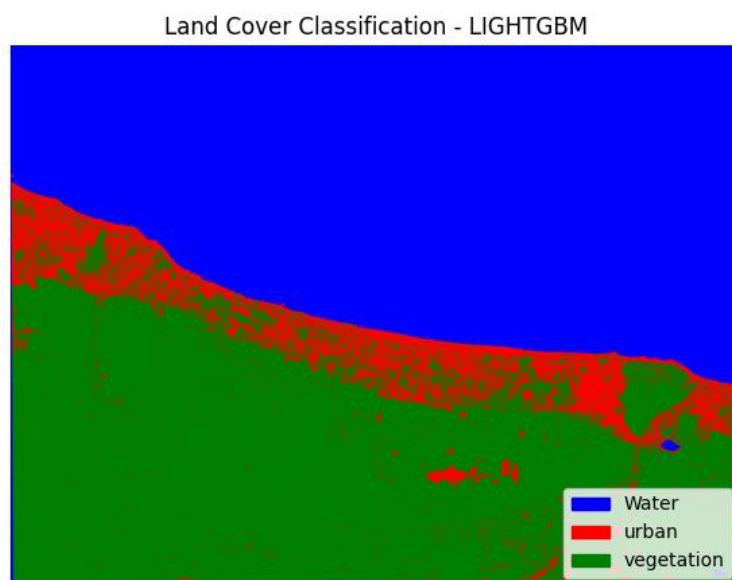
`n_estimators`: تعداد درخت‌هایی که در مدل ساخته می‌شوند.

`learning_rate`: نرخ یادگیری.

`random_state`: تعداد تکرارهای مدل.



شکل ۴-۱) کلاسه‌بندی LightGBM با داده‌های سطح Low



شکل ۴-۲) کلاسه‌بندی LightGBM با داده‌های سطح Medium



شکل ۴-۳) کلاسه‌بندی LightGBM با داده‌های سطح High

مقدار کاپا، گزارش کلاسه بندی و ماتریس کانفیوژن ما برای داده‌های سطح Low، Medium و High به ترتیب براساس شکل‌های ۴-۴، ۴-۵ و ۴-۶ و جدول‌های ۴-۱، ۴-۲ و ۴-۳ هستند.

Kappa lightgbm: 0.9				
classification report lightgmb:				
	precision	recall	f1-score	support
0	0.94	1.00	0.97	15
1	0.88	0.93	0.90	15
2	1.00	0.87	0.93	15
accuracy			0.93	45
macro avg	0.94	0.93	0.93	45
weighted avg	0.94	0.93	0.93	45

شکل ۴-۴) کاپا و گزارش کلاسه‌بندی LightGBM برای داده سطح Low

جدول ۴-۱) ماتریس کانفیوژن برای کلاسه‌بندی LightGBM برای داده‌های Low

۱۵	۰	۰
۰	۱۳	۲
۰	۲	۱۳

Kappa lightgbm: 0.85				
classification report lightgbm:				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	30
1	0.86	0.83	0.85	30
2	0.84	0.87	0.85	30
accuracy			0.90	90
macro avg	0.90	0.90	0.90	90
weighted avg	0.90	0.90	0.90	90

شکل ۴-۵) کاپا و گزارش کلاسه‌بندی LightGBM برای داده سطح Medium

جدول ۴-۲) ماتریس کانفیوژن برای کلاسه‌بندی LightGBM برای داده‌های Medium

۳۰	۰	۰
۰	۲۵	۵
۰	۴	۲۶

Kappa lightgbm: 0.8876651982378855				
classification report lightgmb:				
	precision	recall	f1-score	support
0	0.98	1.00	0.99	60
1	0.89	0.94	0.91	84
2	0.93	0.83	0.88	60
accuracy			0.93	204
macro avg	0.93	0.92	0.93	204
weighted avg	0.93	0.93	0.93	204

شکل ۴-۶) کاپا و گزارش کلاسه‌بندی LightGBM برای داده سطح High

جدول ۴-۳) ماتریس کانفیوژن برای کلاسه‌بندی LightGBM برای داده‌های High

۶۰	۰	۰
۱	۷۹	۴
۰	۱۰	۵۰

بر اساس گزارش کلاسه‌بندی، پارامترهای زیر برآورد شد (جدول ۴-۴).

جدول ۴-۴) پارامترهای طبقه‌بندی LightGBM

	OA	PA(water)	UA(water)	PA(urban)	UA(urban)	PA(veg)	UA(veg)
High	۰/۹۳	۱	۰/۹۸	۰/۹۴	۰/۸۹	۰/۸۳	۰/۹۳
Medium	۰/۹	۱	۱	۰/۸۳	۰/۸۶	۰/۸۷	۰/۸۴
Low	۰/۹۳	۱	۰/۹۴	۰/۹۳	۰/۸۸	۰/۸۷	۱

۵- مقایسه SVM و Random forest

در تمرین قبل، ما الگوریتم پیادسازی SVM را آموختیم. در سوال ۲ هدف مقایسه دو روش Random Forest و SVM برای ۳ لول از داده‌های ما است.

پیاده‌سازی الگوریتم SVM نیز مانند روش Random Forest بوده اما تابع آن SVC است.

```
svm_rbf=SVC(kernel='rbf' , C=1 , gamma=0.1)
svm_rbf.fit(x_train_s , y_train)

y_pred_svm= svm_rbf.predict(x_test_s)
pred_rbf_full=svm_rbf.predict(flat_s).reshape(rows, cols)
```

پارامترهای تعریف شده برای این روش به شرح زیر است:

kernel: پارامتر برای نگاشت داده ها به فضای با ابعاد بالاتر.

C: میزان جریمه خطا.

gamma: تعیین فاصله تاثیر هر نمونه آموزشی.

پارامترهایی که باید برای Random Forest در نظر بگیریم به شرح زیر است:

n_estimators=300

max_depth=15

همچنین پارامترهای روش SVM نیز به شرح زیر می باشد:

Kernel=RBF

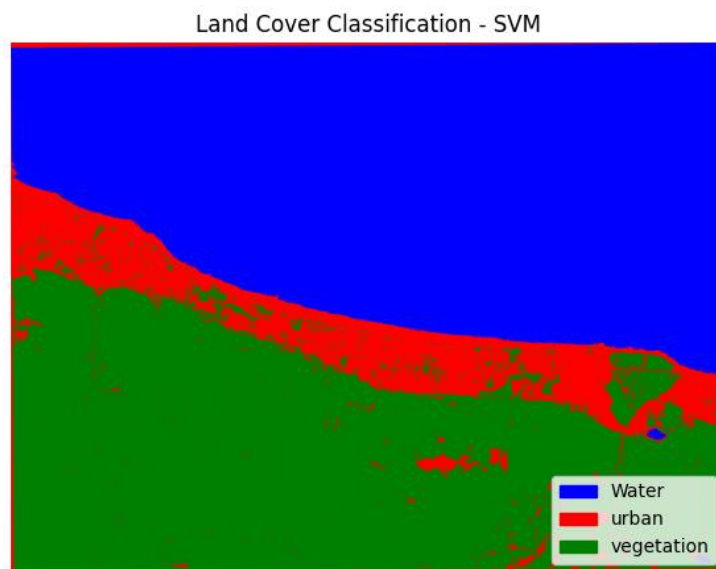
C=10

gamma=0.1

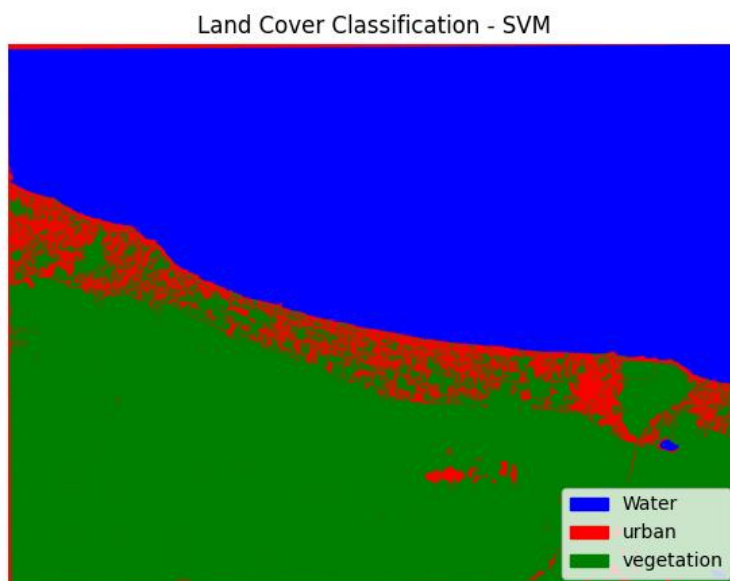
این طبقه بندی را با استفاده از هر سه نوع داده های آموزشی انجام می دهیم.

تصاویر طبقه بندی SVM مانند شکل های ۱-۵، ۲-۵ و ۳-۵ است. همچنین تصاویر طبقه بندی Random

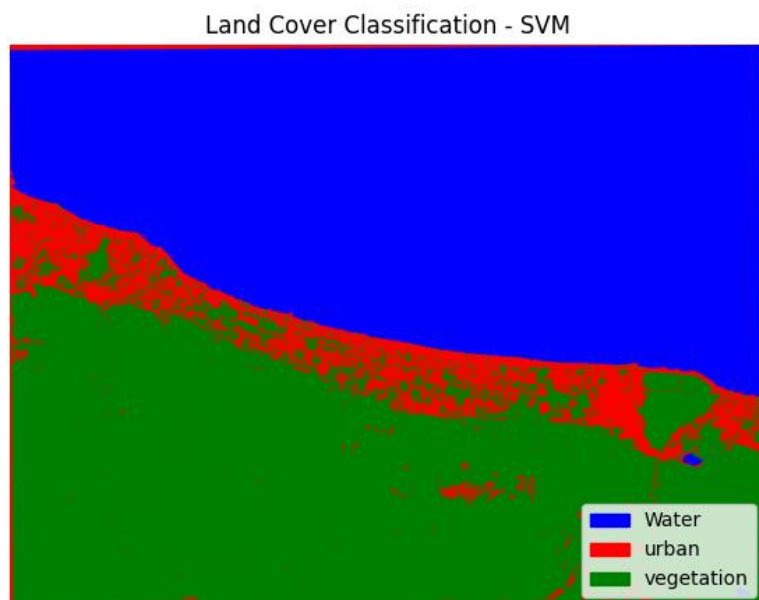
Forest مانند شکل های ۴-۵، ۵-۵ و ۶-۵ می باشد.



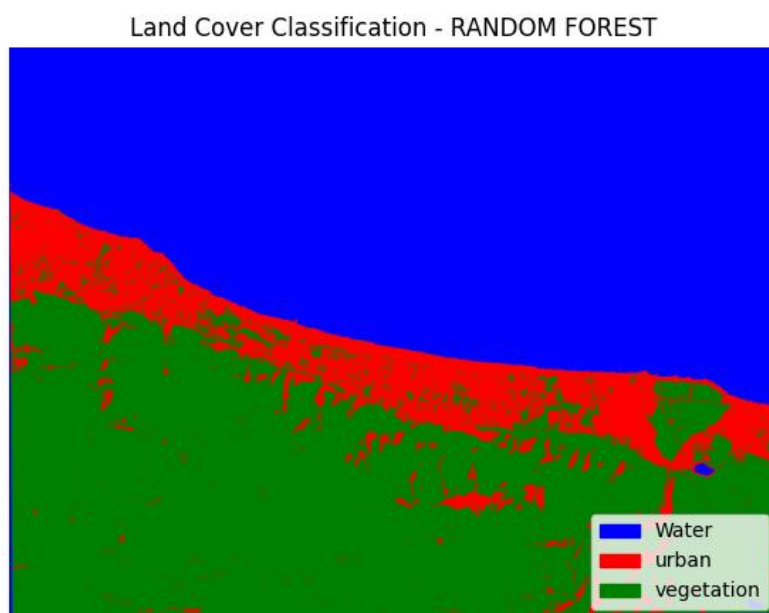
شکل ۵-۱) طبقه‌بندی SVM با داده‌های سطح Low



شکل ۵-۲) طبقه‌بندی SVM با داده‌های سطح Medium

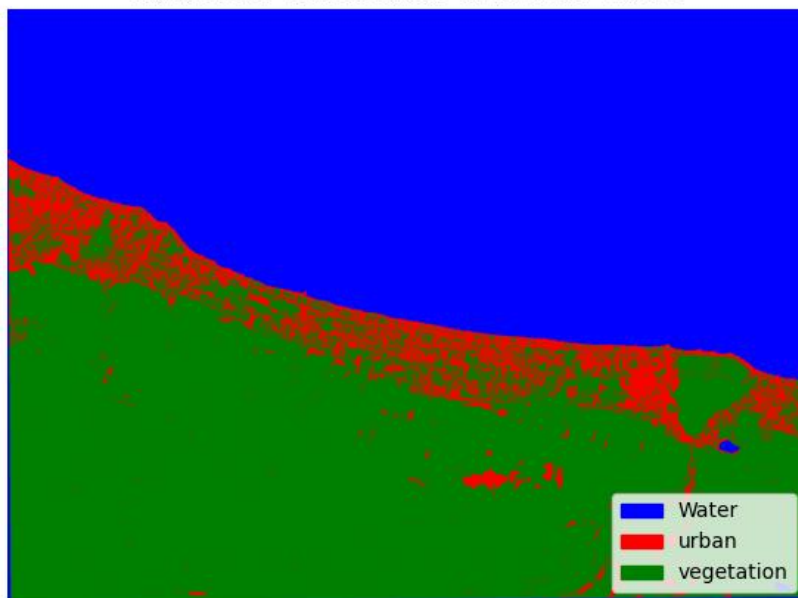


شکل ۳-۵) طبقه‌بندی SVM با داده‌های سطح High



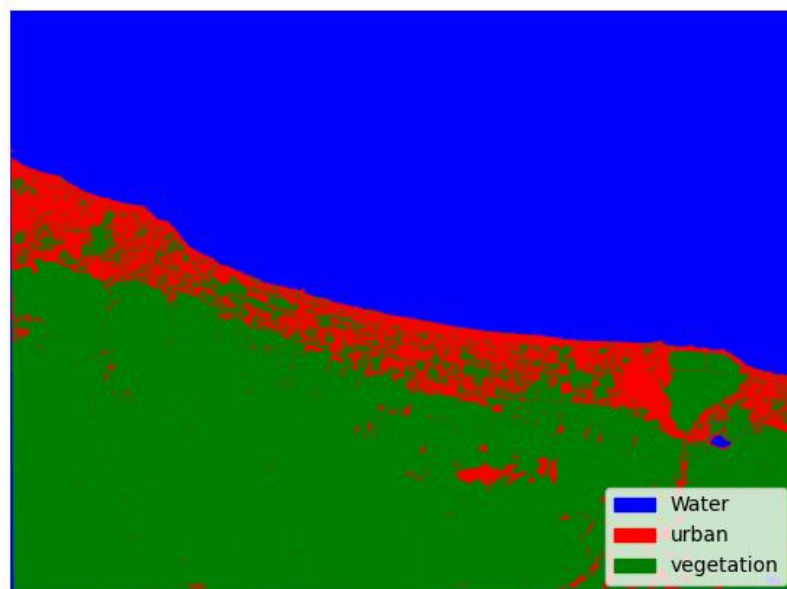
شکل ۴-۵) طبقه‌بندی Random Forest با داده‌های سطح Low

Land Cover Classification - RANDOM FOREST



شکل ۵-۵ طبقه‌بندی Random Forest با داده‌های سطح Medium

Land Cover Classification - RANDOM FOREST



شکل ۵-۶ طبقه‌بندی Random Forest با داده‌های سطح High

در هر دو روش، ابتدا مقدار kappa را تعیین و سپس با استفاده از گزارش کلاسه‌بندی که تولید می‌کنیم، مقادیر

OA، PA، UA خود را حساب می‌کنیم. در جدول ۵-۱، این ۴ مقدار برای دو روش SVM و Random

Forest برای هر سه سطح داده مقایسه شده‌است.

جدول ۵-۱) پارامترهای مقایسه دو روش برای سه سطح داده

	OA	PA(water)	UA(water)	PA(urban)	UA(urban)	PA(veg)	UA(veg)	Kappa
High RF	۰/۹۵	۱	۰/۹۸	۰/۹۵	۰/۹۲	۰/۸۸	۰/۹۵	۰/۹۱۸
High SVM	۰/۹۶	۱	۰/۹۸	۰/۹۵	۰/۹۴	۰/۹۲	۰/۹۵	۰/۹۳۳
Medium RF	۰/۹۳	۱	۱	۰/۸۷	۰/۹۳	۰/۹۳	۰/۸۸	۰/۹
Medium SVM	۰/۹۳	۱	۱	۰/۸۳	۰/۹۶	۰/۹۷	۰/۸۵	۰/۹
Low RF	۰/۹۳	۱	۱	۰/۸۷	۰/۹۳	۰/۹۳	۰/۸۸	۰/۹
Low SVM	۰/۹۳	۱	۱	۰/۸۷	۰/۹۳	۰/۹۳	۰/۸۸	۰/۹

بر اساس جدول ۵-۱، هر دو الگوریتم Random Forest و SVM در سطح High عملکرد مناسبی داشته‌اند،

به‌طوری‌که بیشترین مقدار دقت کلی مربوط به روش High SVM با مقدار ۰/۹۶ می‌باشد. الگوریتم Random

Forest نیز تقریباً همین دقت را داشته و دقت کلی برابر با ۰/۹۶ را نشان داده که اختلاف کمی با SVM دارد.

بررسی دقت‌ها برای کلاس آب نشان می‌دهد که مقدار PA برای این آن در تمامی روش‌ها برابر با ۱ می‌باشد که بیانگر تشخیص همه پیکسل‌های مربوط به این کلاس بر اساس داده‌های آموزشی و تست ما می‌باشد. همچنین مقادیر بالای UA برای کلاس آب نشان می‌دهد که پیکسل‌هایی که به عنوان آب طبقه‌بندی شده‌اند، با دقت بالایی درست بوده‌اند و فقط زمانی که داده‌های آموزشی زیادی را وارد می‌کنیم دقت کاهش می‌یابد.

برای کلاس مناطق شهری، مقادیر PA و UA نسبت به کلاس آب کمتر شده‌است. به‌ویژه در سطوح داده Low و Medium، مقدار PA کاهش یافته‌است. همچنین با توجه به جدول، مشخص است که مقدار PA در سطح داده High بهتر بوده اما مقدار UA در هر سه سطح تقریباً یکسان است.

در مورد کلاس پوشش گیاهی، نتایج نشان می‌دهد که نسبت به کلاس آب دقت کمتری مشاهده می‌شود و در این پوشش، High SVM بهترین عملکرد را داشته است. به طور کلی، هر دو الگوریتم در طبقه‌بندی این کلاس عملکرد نسبتاً مناسبی داشته‌اند. در نهایت، مقادیر ضریب کاپا نشان می‌دهد که میزان دقت طبقه‌بندی با توجه به داده‌های مرجع در تمامی روش‌ها، بالا بوده‌است. با این حال، با کاهش سطح کیفیت داده‌ها از High به پایین‌تر، مقدار کاپا و دقت کلی کاهش می‌یابد.

به طور کلی می‌توان گفت که الگوریتم SVM در سطح High عملکرد بهتری داشته، در حالی که الگوریتم Random Forest در سطح Medium بهتر بوده و در سطح Low عملکرد هر دو مشابه بوده‌است.

۶- بررسی overfitting

Overfitting، زمانی اتفاق می‌افتد که مدل یادگیری ماشین بیش از حد به داده‌های آموزشی منطبق می‌شود و در نتیجه قادر به تطابق با داده‌های جدید نیست. در این حالت، مدل علاوه بر ویژگی‌های واقعی داده‌ها، نویز و جزئیات خاص داده‌های آموزشی را نیز حفظ می‌کند. این باعث می‌شود که مدل در داده‌های تست عملکرد ضعیفی داشته‌باشد. برای جلوگیری از overfitting می‌توان از تکنیک‌هایی مثل کاهش پیچیدگی مدل Regularization و یا توقف پیش از پایان آموزش استفاده کرد.

در این مرحله از تصویر قصد داریم ۴ مدل مختلف را روی داده‌های آموزشی سطح High خود پیاده‌سازی کنیم و بررسی کنیم که آیا overfitting رخ داده‌است یا خیر. دو مدل از این ۴ مدل ما روش Random Forest بوده و دو مدل روش SVM که پارامترهای متفاوتی دارند که طبق جدول ۱-۶ می‌باشد. برای بررسی overfitting، باید گزارش کلاسه‌بندی برای داده‌های آموزشی خود را نیز تهیه کنیم. برای این کار، پنج خط کد زیر را به دو الگوریتم خود اضافه می‌کنیم که بر اساس داده‌های تست ما نیز کلاسه‌بندی انجام شود.

```
y_train_pred = rf.predict(x_train_s)
print("classification report for test random forest:")
print(classification_report(y_train, y_train_pred))
kappa1 = cohen_kappa_score(y_train, y_train_pred)
print("Kappa random forest:", kappa1)
```

```
y_train_pred = svm_rbf.predict(x_train_s)
print("classification report for test svm:")
print(classification_report(y_train, y_train_pred))
kappa1 = cohen_kappa_score(y_train, y_train_pred)
print("Kappa random forest:", kappa1)
```

جدول ۶-۱) مدل‌های مورد آزمایش برای بررسی overfitting

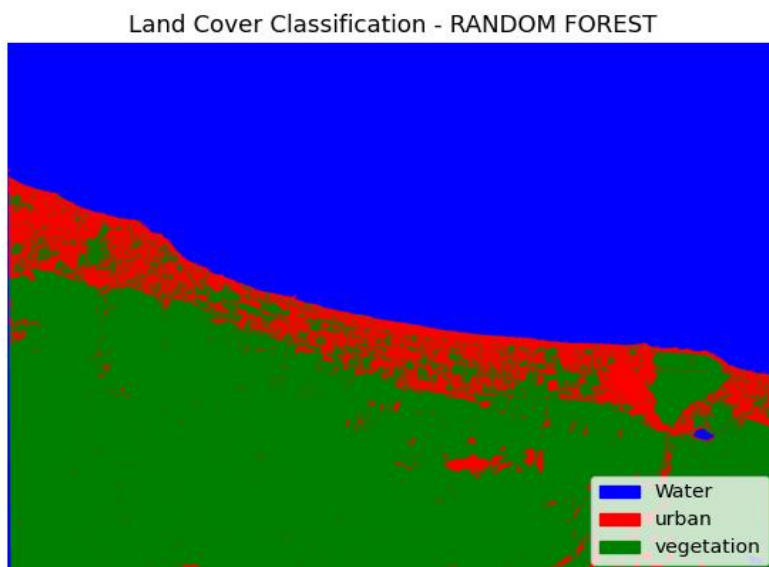
مدل	پارامترها
RF	n_estimators=300 , max_depth=None
RF	n_estimators=200 , max_depth=15
SVM	Kernel=RBF , C=100 , gamma=1
SVM	Kernel=RBF , C=10 , gamma=0.1

مدل اول

در ابتدا مدل اول را بررسی می‌کنیم. مدل اول ما، الگوریتم Random Forest بوده که در آن n_estimators خود را روی ۳۰۰ قرار داده و پارامتر max_depth را حذف می‌کنیم.

طبقه‌بندی ما بر اساس شکل ۶-۱ بوده و گزارش کلاسه‌بندی، ماتریس کانفیوژن و کاپا ما طبق شکل ۶-۲ می‌باشد.

شکل ۶-۱) نتیجه طبقه‌بندی مدل اول



شکل ۲-۶) گزارش کلاسه‌بندی، کاپا و ماتریس کانفیوژن مدل اول

```
classification report for test random forest:
              precision    recall  f1-score   support

     1         1.00      1.00      1.00     140
     2         1.00      1.00      1.00     196
     3         1.00      1.00      1.00     140

 accuracy          1.00      1.00      1.00     476
 macro avg          1.00      1.00      1.00     476
weighted avg          1.00      1.00      1.00     476

Kappa random forest: 1.0
classification report random forest:
              precision    recall  f1-score   support

     1         0.98      1.00      0.99      60
     2         0.92      0.95      0.94      84
     3         0.95      0.88      0.91      60

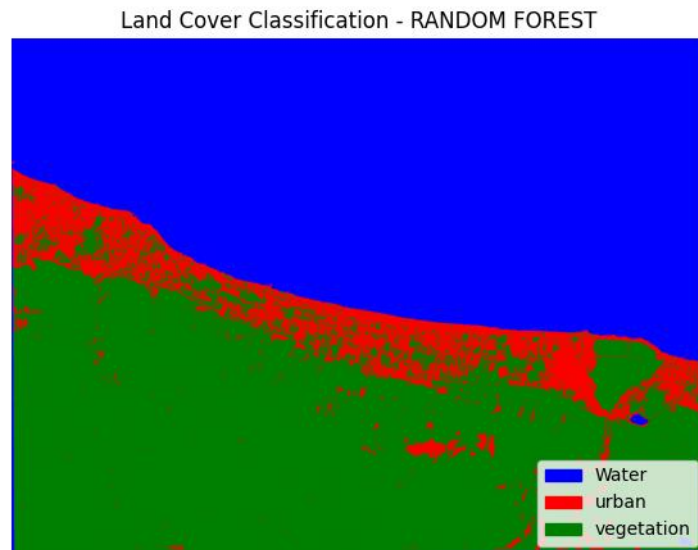
 accuracy          0.95      0.95      0.95     204
 macro avg          0.95      0.95      0.95     204
weighted avg          0.95      0.95      0.95     204

confusion matrix random forest:
[[60  0  0]
 [ 1 80  3]
 [ 0  7 53]]
Kappa random forest: 0.9177660510114336
```

مدل دوم

سپس، طبقه‌بندی با استفاده از مدل دوم را انجام می‌دهیم که مانند کدل اول Random Forest است. در این مدل n_estimators خود را ۲۰۰ و max_depth را ۱۵ قرار می‌دهیم. طبقه‌بندی ما بر اساس شکل ۳-۶ بوده و گزارش کلاسه‌بندی، ماتریس کانفیوژن و کاپا ما طبق شکل ۴-۶ می‌باشد.

شکل ۳-۶) نتیجه طبقه‌بندی مدل دوم



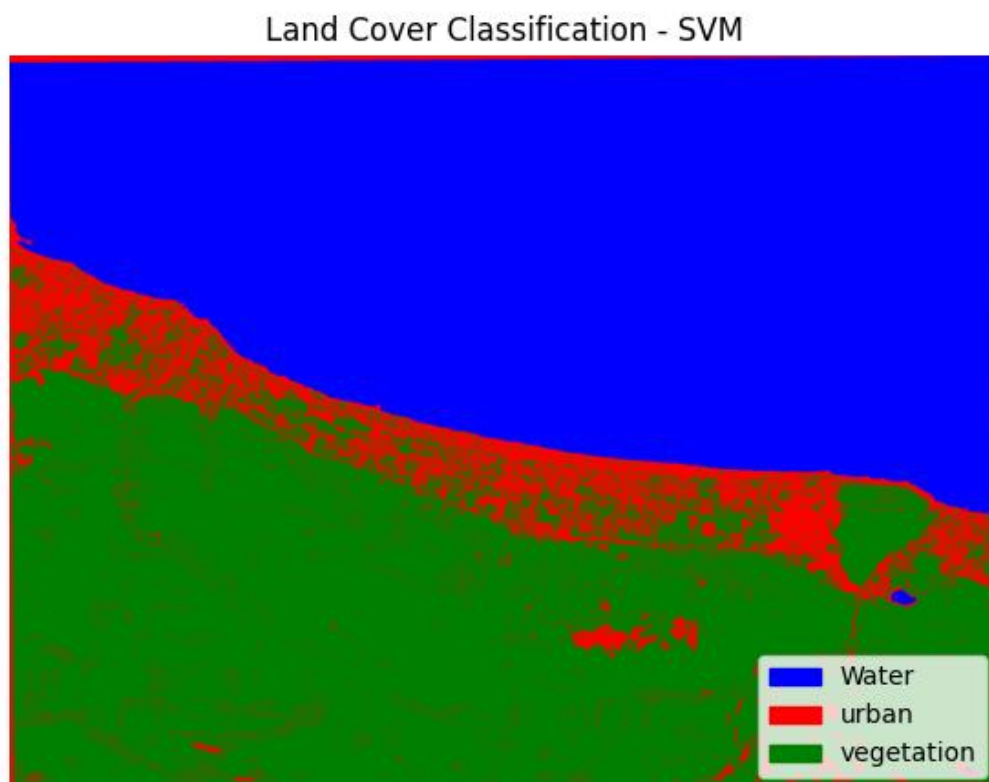
شکل ۴-۶) گزارش کلاسه‌بندی، کاپا و ماتریس کانفیوژن مدل دوم

classification report for test random forest:				
	precision	recall	f1-score	support
1	1.00	1.00	1.00	140
2	1.00	1.00	1.00	196
3	1.00	1.00	1.00	140
accuracy			1.00	476
macro avg	1.00	1.00	1.00	476
weighted avg	1.00	1.00	1.00	476
Kappa random forest: 1.0				
classification report random forest:				
	precision	recall	f1-score	support
1	0.98	1.00	0.99	60
2	0.92	0.95	0.94	84
3	0.95	0.88	0.91	60
accuracy			0.95	204
macro avg	0.95	0.95	0.95	204
weighted avg	0.95	0.95	0.95	204
confusion matrix random forest:				
[[60 0 0]				
[1 80 3]				
[0 7 53]]				
Kappa random forest: 0.9177660510114336				

مدل سوم

در مرحله بعد، طبقه‌بندی با استفاده از مدل سوم را انجام می‌دهیم. در این مدل که برخلاف دو مدل قبلی که Random Forest بودند، SVM است، کرنل ما RBF، C برابر ۱۰۰ و gamma ما ۱ می‌باشد. طبقه‌بندی ما بر اساس شکل ۵-۶ بوده و گزارش کلاسه‌بندی، ماتریس کانفیوژن و کاپا ما طبق شکل ۶-۶ می‌باشد.

شکل ۵-۶) نتیجه طبقه‌بندی مدل سوم



شکل ۶-۶ گزارش کلاسه‌بندی، کاپا و ماتریس کانفیوژن مدل سوم

```

classification report for test svm:
              precision    recall  f1-score   support

     1         1.00        1.00        1.00        140
     2         1.00        0.99        1.00        196
     3         0.99        1.00        1.00        140

 accuracy         1.00        1.00        1.00        476
 macro avg         1.00        1.00        1.00        476
weighted avg         1.00        1.00        1.00        476

Kappa random forest: 0.9968057121382938
classification report svm:
              precision    recall  f1-score   support

     1         1.00        1.00        1.00         60
     2         0.87        0.90        0.89         84
     3         0.86        0.82        0.84         60

 accuracy         0.91        0.91        0.91        204
 macro avg         0.91        0.91        0.91        204
weighted avg         0.91        0.91        0.91        204

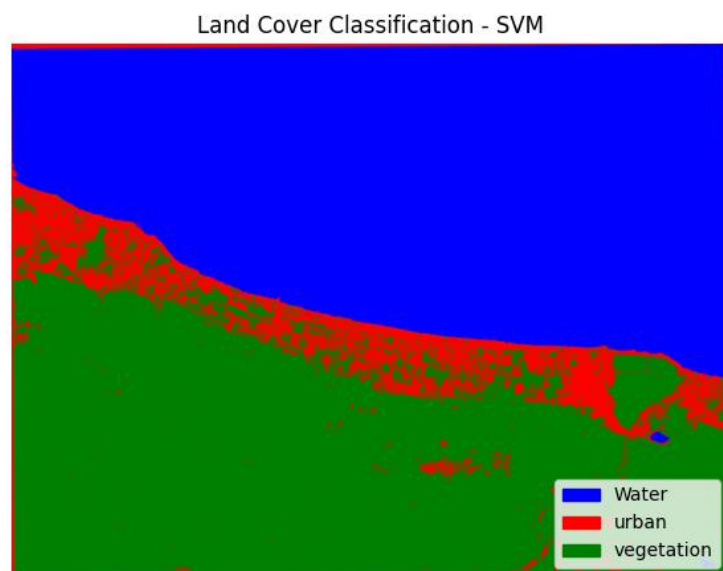
confusion matrix svm:
[[60  0  0]
 [ 0 76  8]
 [ 0 11 49]]
Kappa svm: 0.8579595426561126

```

مدل چهارم

در نهایت، طبقه‌بندی خود را با استفاده از مدل چهارم انجام می‌دهیم. در این مدل که مانند مدل قبلی، SVM است، کرنل ما RBF، C برابر ۱۰ و gamma ما ۰/۱ می‌باشد. طبقه‌بندی ما بر اساس شکل ۶-۷ بوده و گزارش کلاسه‌بندی، ماتریس کانفیوژن و کاپا ما طبق شکل ۶-۸ می‌باشد.

شکل ۵-۶) نتیجه طبقه‌بندی مدل سوم



شکل ۸-۶) گزارش کلاسه‌بندی، کاپا و ماتریس کانفیوژن مدل چهارم

```

classification report for test svm:
              precision    recall  f1-score   support

     1         1.00        1.00        1.00        140
     2         0.97        0.94        0.96        196
     3         0.92        0.96        0.94        140

 accuracy          0.96
 macro avg         0.96
 weighted avg      0.96

Kappa random forest: 0.9457786116322702
classification report svm:
              precision    recall  f1-score   support

     1         0.98        1.00        0.99         60
     2         0.94        0.95        0.95         84
     3         0.95        0.92        0.93         60

 accuracy          0.96
 macro avg         0.96
 weighted avg      0.96

confusion matrix svm:
[[60  0  0]
 [ 1 80  3]
 [ 0  5 55]]
Kappa svm: 0.9328358208955224
    
```

با توجه به جدول‌های بالا می‌توان تشخیص داد که در مدل سوم به احتمال بالا overfitting رخ داده‌است. چون دقت کلی کلاسه‌بندی داده‌های تست ما تقریباً ۱۰ درصد کمتر بوده و همچنین کاپای ما حدوداً ۱۵ درصد کاهش یافته که می‌تواند از نشانه‌های overfitting باشد.

در مدل‌های ۱ و ۲ احتمال overfitting وجود دارد اما کم است. چون تفاوت بین دقت کلی کلاسه‌بندی در دو نوع داده آموزشی و تست در دو مدل کم بوده و همچنین تفاوت کاپای هر دو کمتر از ۱۰ درصد است، می‌تواند طبیعی باشد.

در مدل ۴ دقت کلی و کاپا در هر دو یکسان بوده و در نتیجه بیش‌برازش نداریم. اما روش ۴ با توجه به داده‌های ما مناسب نیست. چون آموزش ما باید با دقت ۱۰۰ درصد انجام شود که در این حالت نشده‌است.